

“Database Client-Server Management System”

A Project Report Submitted in the partial fulfillment of the requirements for the award of the degree of

BACHELOR OF TECHNOLOGY

In

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING

By

MYDAM ASWITHA PATEL 2520030001
NAGABHYRAVA YISHNAANI 2520030414

Under the Esteemed Guidance of

Dr. K. Hema

Assistant Professor

Department of Computer Science and Engineering



Koneru Lakshmaiah Education Foundation

(Deemed to be University estd. u/s. 3 of the UGC Act, 1956)

Off-Campus: Bachupally-Gandimaisamma Road, Bowrampet, Hyderabad, Telangana - 500 043.

Phone No: 7815926816, www.klh.edu.in

**K L (Deemed to be) University
DEPARTMENT OF COMPUTER SCIENCE ENGINEERING**

&

DEPARTMENT OF COMPUTER SCIENCE AND INFORMATION TECHNOLOGY



Declaration

The Project Report entitled **“Database-Client-Server-Management-System”** is a record of Bonafide work of **MYDAM ASWITHA PATEL - 2520030001, NAGABHYRAVA YISHNAANI - 2520030414**, submitted in partial fulfillment for the award of B. Tech in Computer Science and Engineering (or) Computer Science and Information Technology to the K L University. The results embodied in this report have not been copied from any other departments/University/Institute.

MYDAM ASWITHA PATEL 2520030001
NAGABHYRAVA YISHNAANI 2520030414

K L (Deemed to be) University

DEPARTMENT OF COMPUTER SCIENCE ENGINEERING



CERTIFICATE

This is certify that the mini project-based report entitled **“Database-Client-Server-Management-System”** is a bonafide work done and submitted by **Mydam Aswitha Patel – 2520030001**, **Nagabhyrava Yishnaani - 2520030414** in partial fulfillment of the requirements for the award of the degree of **BACHELOR OF TECHNOLOGY** in Department of Computer Science Engineering, K L (Deemed to be University), during the academic year **2024-2025**.

Signature of the Guide

Signature of the Course Coordinator

Signature of the HOD

ACKNOWLEDGEMENT

The success in this project would not have been possible but for the timely help and guidance rendered by many people. Our wish to express my sincere thanks to all those who has assisted us in one way or the other for the completion of my project.

Our greatest appreciation to my Course Coordinator **Dr. K. Hema**, and my guide **Dr. K. Hema**, Department of Computer Science which cannot be expressed in words for his/her tremendous support, encouragement and guidance for this project.

We express our gratitude to **Dr. P Venkateshwara Roa (CSE)**, Head of the *Department for Computer Science Engineering* for providing us with adequate facilities, ways and means by which we are able to complete this project-based Lab.

We thank all the members of teaching and non-teaching staff members, and also who have assisted me directly or indirectly for successful completion of this project. Finally, I sincerely thank my parents, friends and classmates for their kind help and cooperation during my work.

MYDAM ASWITHA PATEL 2520030001

NAGABHYRAVA YISHNAANI 2520030414

Abstract

This project proposes a Linux-based database client-server system in C for managing a centralized university student database containing student information, marks, attendance, and course records. The system is designed to handle multiple clients accessing shared data simultaneously while preventing data corruption and maintaining consistency. The project applies key Operating Systems and System Programming concepts. The server uses Linux system calls and kernel services for system and file operations. Multiple client requests are handled using processes and POSIX threads, with worker threads processing requests concurrently. A shared request and response queue is used for communication, while signals and POSIX signal handlers support configuration reload and controlled server shutdown.

Mutexes, condition variables, and counting semaphores are used to coordinate threads, protect shared database structures, and control access to shared resources. The system supports insert, search, update, and delete operations. Student records are managed using file abstractions, file descriptors, and Linux file I/O system calls. Memory-mapped file I/O is used for efficient access to frequently used data. Transaction and server activity logs are maintained, and active client connections and server resources are monitored.

The expected outcome is a reliable and efficient concurrent database server demonstrating practical applications of process control, IPC, signals, memory management, file I/O, POSIX threads, and synchronization.

Index

S. No.	Chapter	Topics / Subtopics	Page No.
	Acknowledgement		i
	Abstract		ii
1	Introduction		8–12
		1.1 Background of the Project	
		1.2 Problem Statement	
		1.3 Scope of the Project	
		1.4 Objectives of the Project	
		1.5 Importance of the Application	
		1.6 Target Users / Audience	
2	OS Concepts and Background Study		13–15
		2.1 Operating System Concepts Related to the Project	
		2.2 Existing Methods / Algorithms	
		2.3 Proposed Approach	
		2.4 Advantages of the Proposed Approach	
3	System Requirements		16–17
		3.1 Hardware Requirements	
		3.2 Software Requirements	
		3.3 Operating System Environment – Ubuntu / WSL	
		3.4 Development Tools and Compilers	
4	Technology and Tools Used		18–19
		4.1 C / C++ Programming	
		4.2 GCC Compiler	
		4.3 Linux System Calls	
		4.4 POSIX Threads / Synchronization	
		4.5 Linux Commands and Utilities	
		4.6 Other Tools / Libraries Used	
5	System Architecture and Design		20–25
		5.1 Overall System Architecture	
		5.2 System Workflow	
		5.3 Flowchart / Data Flow	

		5.4 Module Description	
		5.5 Input and Output Design	
6	Implementation		26–32
		6.1 Module-wise Implementation	
		6.2 Algorithm / Pseudocode	
		6.3 Core C / C++ Implementation	
		6.4 Linux System Calls / APIs Used	
		6.5 Process / Thread Implementation	
		6.6 IPC / Synchronization Implementation	
		6.7 Memory Management Implementation	
		6.8 Optional User Interface Implementation	
7	Execution and Demonstration		33–39
		7.1 Compilation in Ubuntu	
		7.2 Execution Commands	
		7.3 Sample Inputs and Outputs	
		7.4 Screenshots of Execution	
		7.5 Demonstration of OS Concepts	
8	Conclusion		40–42
		8.1 Summary of the Project	
		8.2 Objectives Achieved	
		8.3 Results Achieved	
		8.4 Skills and Learning Outcomes	
9	References		43–44
		9.1 Books	
		9.2 Research Papers	
		9.3 Linux / GNU Documentation	
		9.4 Online Resources	

CHAPTER -1 INTRODUCTION

1.1 Background of the Project

In today's educational institutions, a large amount of student information such as personal details, marks, attendance, and course details needs to be stored and managed regularly. When this information is handled manually or through simple file-based methods, it can become difficult to search, update, and maintain the records efficiently. It can also lead to data duplication, delays, and difficulties when multiple users access the system at the same time.

The Database Client-Server Management System is developed to provide an efficient and organized way to manage student records. The system follows a client-server architecture, where multiple clients can connect to a central server and perform operations such as inserting, searching, updating, deleting, and displaying student records.

The project focuses on important Operating System concepts such as multithreading, synchronization, mutexes, semaphores, inter-process communication, memory mapping, signals, and resource monitoring. Multiple worker threads are used by the server to handle client requests concurrently, while synchronization mechanisms protect shared data from conflicts.

The student database is stored using memory-mapped files (mmap), which allows the server to access data efficiently. Request and response queues are used to manage communication between clients and worker threads. Logging and monitoring features help track server activities, active clients, worker threads, and system resource usage.

The system also includes user authentication, allowing users to register and log in before accessing database operations. A web-based dashboard provides a simple interface for performing database operations and viewing student information, analytics, attendance alerts, recent activities, and system health.

The project therefore combines client-server communication, multithreading, synchronization, memory management, file handling, authentication, logging, signal handling, and system monitoring into one practical application. It demonstrates how Operating System concepts can be applied to solve real-world problems involving concurrent users and shared resources.

Overall, the project demonstrates how Operating System concepts can be applied to build a practical, secure, concurrent, and efficient database management system for handling student records. The project provides practical understanding of how servers manage multiple clients, how threads coordinate with each other, how shared resources are protected, and how system resources can be monitored in a real-world application.

1.2 Problem Statement

Managing student information in educational institutions involves handling a large number of records such as student ID, name, course, marks, and attendance. When these records are maintained manually or through simple file-based systems, it becomes difficult to efficiently store, search, update, and delete information. Manual management can also result in data duplication, incorrect records, loss of information, and difficulty in maintaining updated student details.

Another major problem occurs when multiple users or clients try to access the database at the same time. A basic database system may not be able to handle concurrent requests properly. If two or more clients attempt to modify shared data simultaneously, it can lead to inconsistent data, race conditions, or database corruption. Therefore, a proper mechanism is required to manage multiple client requests and provide controlled access to shared resources.

The system also needs an efficient way to process client requests without making every client wait for the previous request to finish. A single-threaded server may become slow when several clients connect simultaneously. Hence, there is a need for a multithreaded client-server system in which multiple worker threads can process different requests concurrently.

Since the database and other resources are shared among multiple threads, synchronization is another important requirement. Without proper synchronization, simultaneous operations can interfere with each other and produce incorrect results. Therefore, mechanisms such as mutexes and semaphores are required to control access to shared resources and maintain data consistency.

Efficient database access is also necessary for handling student records. Continuously performing file input/output operations can reduce performance when the database becomes larger. Therefore, the system requires an efficient memory management technique such as memory mapping (mmap) to provide faster access to the stored database records.

In addition, the server should provide proper authentication and access control so that only registered and authenticated users can perform database operations. There is also a need to maintain logs of database transactions and server activities so that important operations can be tracked and problems can be identified when required.

Monitoring the server is another challenge. Administrators need to know information such as the number of active clients, number of worker threads, total threads, and memory usage. Without proper monitoring, it becomes difficult to understand the server's current state and resource utilization. The system should also support controlled shutdown and configuration updates using signals.

Therefore, the main problem is to develop a reliable, concurrent, and efficient Database Client-Server Management System that can manage student records while supporting multiple clients simultaneously. The system must provide CRUD operations, authentication, multithreading, synchronization, memory mapping, logging, signal handling, and resource monitoring.

The proposed system aims to overcome the limitations of basic file-based student management systems and provide a practical application that demonstrates important Operating System concepts in a real-world environment.

1.3 Scope of the project

The scope of the Database Client-Server Management System is to provide an efficient platform for storing, accessing, and managing student records through a centralized server. The system is designed to support multiple clients and demonstrate important Operating System concepts.

The major scope of the project includes:

- **Student Record Management:** Provides operations to insert, search, update, delete, and display student records
- **Client-Server Communication:** Allows multiple clients to communicate with a central server and send database requests.
- **Multithreading:** Uses multiple worker threads to process client requests concurrently.
- **Synchronization:** Uses mutexes and semaphores to safely manage shared resources and prevent data conflicts.
- **Memory Management:** Uses memory mapping for efficient access to the student database.
- **Authentication:** Provides user registration and login to control access to the system.
- **Logging and Monitoring:** Maintains server activity logs and displays information about active clients, threads, and memory usage.
- **Web Dashboard:** Provides a simple graphical interface for performing database operations and viewing student information.

Overall, the project focuses on demonstrating how Operating System concepts can be combined with database management and client-server communication to build an efficient and reliable student management system. The project manages student records such as ID, name, course, marks, and attendance. It supports multiple clients through a centralized client-server system. It uses multithreading, mutexes, semaphores, and memory mapping for efficient processing. It provides authentication, CRUD operations, logging, and server monitoring. A web dashboard is provided for easy access and management of student information.

1.4 Objectives of the Project

- To develop a client-server system for managing student records efficiently.
- To implement CRUD operations such as insert, search, update, delete, and display.
- To support multiple clients using multithreading and synchronization.
- To use memory mapping for efficient database access.
- To provide authentication, logging, and system monitoring for secure and reliable operation.
- To develop a simple web dashboard for easy student record management.

1.5 Importance of the Application

The Database Client-Server Management System is important because it provides an organized and efficient way to manage student information. In educational institutions, a large number of student records such as student ID, name, course, marks, and attendance need to be stored and accessed regularly. Managing these records manually can take more time and may result in errors. This application provides a centralized system where student information can be easily stored, searched, updated, deleted, and displayed.

The application is also important because it supports multiple clients at the same time. The client-server architecture allows different users to send requests to a central server, while worker threads process these requests concurrently. This improves the efficiency and responsiveness of the system. The use of mutexes and semaphores ensures that shared resources are accessed safely and prevents conflicts when multiple operations are performed simultaneously.

Another important feature is the use of memory mapping, which provides efficient access to the student database. The application also includes user authentication, helping to ensure that only authorized users can access database operations. Logging records important server and database activities, making it easier to track operations and identify issues.

The system also provides server monitoring, which displays information such as active clients, worker threads, total threads, and memory usage. This helps understand the current performance and resource usage of the server. The web-based dashboard further improves usability by providing a simple interface for managing records and viewing important student information.

Overall, the application is important not only for managing student data efficiently but also for demonstrating the practical implementation of Operating System concepts such as multithreading, synchronization, memory management, client-server communication, authentication, logging, and resource monitoring in a real-world application.

1.6 Target Users / Audience

The Database Client-Server Management System is mainly designed for educational institutions such as colleges and universities where student information needs to be stored, maintained, and accessed regularly. The system provides different users with a simple and organized way to manage student records through a centralized server.

The primary users of the application are college administrators and office staff. They can use the system to add new student records, search for existing records, update student information, delete outdated records, and display complete student details. This reduces the need for maintaining records manually and makes student data easier to manage.

Faculty members can also use the system to access academic information such as marks and attendance. This can help faculty members maintain updated records and quickly find the required information when needed.

The application can also be used by system or database administrators. They can monitor the server and observe important information such as active clients, worker threads, total threads, and memory usage. The logging and monitoring features help administrators understand system activity and identify possible issues.

Students can be considered as secondary users of the system. Based on the access permissions provided by the institution, students can view their academic information such as marks and attendance. This provides easier access to their records without depending completely on manual processes.

The system is also useful for IT and technical staff who are responsible for maintaining the client-server application. They can monitor server performance, manage user access, and check system activities through the available monitoring and logging features.

Overall, the target audience includes administrators, faculty members, system administrators, technical staff, and students. The application is suitable for institutions that require a centralized system for managing student records while supporting multiple users and demonstrating secure and efficient client-server communication.

CHAPTER -2 OS CONCEPTS AND SYSTEM BACKGROUND

2.1 Operating System Concepts Related to the Project

The Database Client-Server Management System uses several important Operating System concepts to provide efficient and reliable communication between clients and the server.

1. Multithreading

The server uses multiple worker threads to handle requests from different clients simultaneously. This improves the performance of the system and prevents one client request from blocking other requests.

2. Process and Client Management

The server manages multiple client connections and keeps track of active clients. This allows the system to handle several users accessing the database at the same time.

3. Synchronization

Mutexes are used to protect shared database resources. They prevent multiple threads from modifying the same resource at the same time and help avoid race conditions.

4. Semaphores

Semaphores are used to control access to shared resources and coordinate the activities of different threads. They help manage the request and response queues safely.

5. Inter-Thread Communication

A shared request/response queue is used for communication between client-handling parts of the server and worker threads. This allows requests to be organized and processed efficiently.

6. Memory Management

The project uses memory mapping (mmap) to access the student database file through memory. This provides efficient data access and demonstrates the practical use of memory management.

7. File Management

Student records, configuration information, and server logs are stored in files. The system performs file operations to maintain and retrieve the required information.

8. Signal Handling

Signals are used to perform important server operations such as configuration reload and controlled shutdown. This allows the server to respond to system events safely.

9. Resource Monitoring

The system monitors resources such as active clients, worker threads, total threads, and memory usage. This helps administrators understand the current state and performance of the server.

10. Synchronization and Concurrency Control

The combination of threads, mutexes, semaphores, and queues allows the server to process multiple requests while maintaining data consistency and avoiding conflicts.

Overall, these Operating System concepts make the application concurrent, efficient, reliable, and suitable for handling multiple clients and shared resources.

2.2 Existing Methods / Algorithms

Traditional student record management systems mainly use manual record keeping or basic file-based methods. In these methods, student information is stored in physical registers, spreadsheets, or simple files. Users manually enter and search for records, which can take more time and may lead to errors or duplicate information.

Some existing methods include:

1. Manual Record Management

Student details are maintained in registers or paper files. Searching and updating records is time-consuming and there is a higher possibility of human errors.

2. File-Based Management

Student information is stored in files such as text or binary files. Basic file operations are used to insert, search, update, and delete records. However, handling multiple users simultaneously can be difficult.

3. Single-Threaded Processing

In a simple server system, requests are processed one at a time. When multiple clients send requests, each client may have to wait until the previous request is completed, reducing system performance.

4. Sequential Searching

In basic file-based systems, records are often searched one by one from the beginning of the file. This method is simple but becomes slower as the number of records increases.

5. Basic Client-Server Approach

Traditional client-server applications allow clients to communicate with a central server, but they may not provide proper synchronization, authentication, resource monitoring, or concurrent request handling.

These existing methods are suitable for small-scale applications but have limitations in concurrent access, performance, security, data consistency, and resource management. The proposed project addresses these limitations by using multithreading, mutexes, semaphores, request queues, memory mapping, authentication, logging, and monitoring.

2.3 Proposed Approach

The proposed system is a Database Client-Server Management System designed to manage student records efficiently and securely. A central server receives requests from multiple clients and processes them using multiple worker threads.

The system provides INSERT, SEARCH, UPDATE, DELETE, and DISPLAY operations for managing student information. A request and response queue is used to organize client requests before they are processed by worker threads.

To safely handle multiple clients, mutexes and semaphores are used for synchronization and to prevent conflicts while accessing shared resources. The student database is stored using memory mapping (mmap) for efficient data access.

The system also provides user authentication so that only authorized users can perform database operations. Server activities and database transactions are recorded through logging, while the monitoring module tracks active clients, worker threads, total threads, and memory usage.

A web-based dashboard is provided to make the system easier to use. It allows users to manage student records and view analytics, attendance alerts, recent activities, and server health information.

Overall, the proposed approach combines client-server communication, multithreading, synchronization, memory management, authentication, logging, and monitoring to create an efficient and reliable student database management system.

2.4 Advantages of the Proposed Approach

The proposed approach provides several advantages over traditional student record management methods:

1. **Efficient Data Management:** Student records can be easily inserted, searched, updated, deleted, and displayed.
2. **Multiple Client Support:** Multiple clients can access the server simultaneously using a multithreaded approach.
3. **Faster Processing:** Worker threads process client requests concurrently, reducing waiting time.
4. **Data Consistency:** Mutexes and semaphores prevent conflicts when multiple threads access shared resources.
5. **Efficient Memory Usage:** Memory mapping provides faster and efficient access to the student database.
6. **Secure Access:** User authentication ensures that only authorized users can access database operations.
7. **Easy Monitoring:** The system provides information about active clients, threads, and memory usage.
8. **User-Friendly Interface:** The web dashboard makes database operations easier and provides a clear view of student records and system status.
9. **Activity Tracking:** Logging helps maintain a record of server and database activities.
10. **Practical OS Implementation:** The project demonstrates important Operating System concepts in a real-world application.

CHAPTER -3 SYSTEM REQUIREMENTS

3.1 Hardware Requirements

The Database Client-Server Management System does not require high-end hardware. It can run on a standard computer or laptop with the following minimum requirements:

Hardware Component	Minimum Requirement
Processor	Intel Core i3 or equivalent
RAM	4 GB
Storage	10 GB free space
Operating System	Ubuntu Linux / Windows with WSL
Network	LAN or Internet connection
Display	Standard monitor
Input Devices	Keyboard and Mouse

The system requires sufficient RAM and processor capacity to run the server, worker threads, database, and web dashboard simultaneously. A stable network connection is required for communication between clients and the server.

3.2 Software Requirements

The following software tools and technologies are required to develop and run the Database Client-Server Management System:

Software	Requirement
Operating System	Ubuntu Linux / Windows with WSL
Programming Language	C
Compiler	GCC
Code Editor	Nano / Visual Studio Code
Build Tool	Make
Web Technologies	HTML, CSS, JavaScript
Web Browser	Google Chrome / Microsoft Edge
Libraries	POSIX Threads (pthread), POSIX Semaphores, Memory Mapping (mmap)
Version Control	Git / GitHub

The system uses C and POSIX system features for the server, database, threading, synchronization, and memory management, while HTML, CSS, and JavaScript are used to provide the web-based dashboard.

3.3 Operating System Environment – Ubuntu / WSL

The project is developed and executed in the Ubuntu Linux environment using Windows Subsystem for Linux (WSL). WSL provides a Linux-based environment within Windows, allowing Linux commands, tools, and system-level programming features to be used without installing a separate operating system.

Ubuntu is used as the main development environment for compiling and running the C client-server application. The GCC compiler is used to compile the source files, while Make is used to build the complete project efficiently.

The Linux environment provides support for important Operating System concepts used in the project, including POSIX threads, mutexes, semaphores, memory mapping (mmap), signals, sockets, file handling, and process/resource monitoring. These features are essential for implementing the concurrent client-server architecture.

WSL also provides access to Linux commands such as gcc, make, nano, ps, and other system utilities. The project database, configuration files, and log files are maintained within the Linux file system.

Thus, Ubuntu/WSL provides a suitable and practical environment for developing, testing, and demonstrating the Operating System concepts implemented in the project.

3.4 Development Tools and Compilers

The project uses various development tools and software to develop, compile, test, and manage the Database Client-Server Management System.

- GCC Compiler: Used to compile the C source files and generate executable programs for the client and server.
- GNU Make: Used to automate the compilation of multiple source files and simplify the build process.
- Nano Editor: Used to create and edit C source files, header files, configuration files, and other project files in the Ubuntu/WSL terminal.
- Visual Studio Code: Can be used for easier editing and viewing of project files.
- Git and GitHub: Used for version control, storing the project, and maintaining different versions of the source code.
- Ubuntu/WSL Terminal: Provides the Linux environment required to compile and execute the project.
- Web Browser: Used to access and test the web-based dashboard of the system.
- GDB: Can be used for debugging C programs and identifying runtime errors.

These tools provide the required environment for coding, compilation, debugging, testing, and execution of the project.

CHAPTER -4 TECHNOLOGY AND TOOLS USED

4.1 C / C++ Programming

The project is mainly developed using the C programming language because it provides direct access to Operating System and system-level programming features. C is used to implement the client-server communication, database operations, worker threads, request queues, synchronization, memory mapping, authentication, logging, and monitoring.

The project uses POSIX libraries available in Linux for concepts such as pthread threads, mutexes, semaphores, sockets, signals, and mmap. These features help the server handle multiple clients efficiently and safely.

The GCC compiler is used to compile the C source files in the Ubuntu/WSL environment. Although the section is titled C/C++ Programming, the implementation of this project is primarily in C, not C++.

4.2 GCC Compiler

GCC (GNU Compiler Collection) is used as the main compiler for developing and building the project. It converts the C source code into executable programs that can run in the Ubuntu/WSL environment.

In this project, GCC is used to compile the client, server, database, queue, logging, authentication, configuration, and monitoring modules. The -pthread option is used to enable POSIX thread support, while other required libraries are linked during compilation.

The project uses Makefile commands to simplify the compilation process. GCC also provides warning options such as -Wall and -Wextra, which help identify possible errors and improve code quality.

Thus, GCC provides a reliable environment for compiling, building, testing, and executing the C-based client-server application.

4.3 Linux System Calls

Linux system calls are used in the project to allow the application to interact directly with the operating system. They provide the basic functionality required for file handling, process management, memory management, networking, and communication.

The project uses system calls and Linux APIs such as open() and close() for file operations, read() and write() for data transfer, and mmap() for mapping the student database file into memory. Socket-related functions are used for communication between clients and the server.

The project also uses pthread APIs for creating and managing worker threads, while mutexes and semaphores are used for synchronization. Signal handling is used to support controlled server shutdown and configuration updates.

These Linux system calls and APIs help the project efficiently manage files, memory, threads, network connections, and system resources while demonstrating important Operating System concepts.

4.4 POSIX Threads / Synchronization

POSIX Threads (Pthreads) are used in the project to support multiple clients and process their requests concurrently. The server creates multiple worker threads, allowing different client requests to be handled without waiting for each request to finish sequentially.

Since multiple threads may access the same database and shared resources, synchronization is required to prevent conflicts and race conditions. The project uses mutexes to provide controlled access to shared database structures and client information.

Semaphores are also used to coordinate access to the request and response queues. They ensure that threads properly wait when resources are unavailable and continue when resources become available.

Thus, POSIX threads and synchronization mechanisms help the system achieve concurrent processing, safe resource sharing, better performance, and data consistency.

4.5 Linux Commands and Utilities

Linux commands and utilities are used throughout the project for development, compilation, execution, testing, and system monitoring. The project is developed in the Ubuntu/WSL environment, where these commands make it easier to manage files and run the client-server application.

Commands such as `cd`, `ls`, `mkdir`, `nano`, and `rm` are used for navigating directories and managing project files. The `gcc` command is used to compile C programs, while `make` is used to build the complete project using the Makefile.

Commands such as `ps`, `top`, and `kill` are useful for monitoring processes and controlling the running server. `cat` is used to view configuration and log files, while `grep` helps search for specific information in files and command output.

These Linux commands and utilities simplify project development and help in file management, compilation, execution, debugging, process management, and system monitoring.

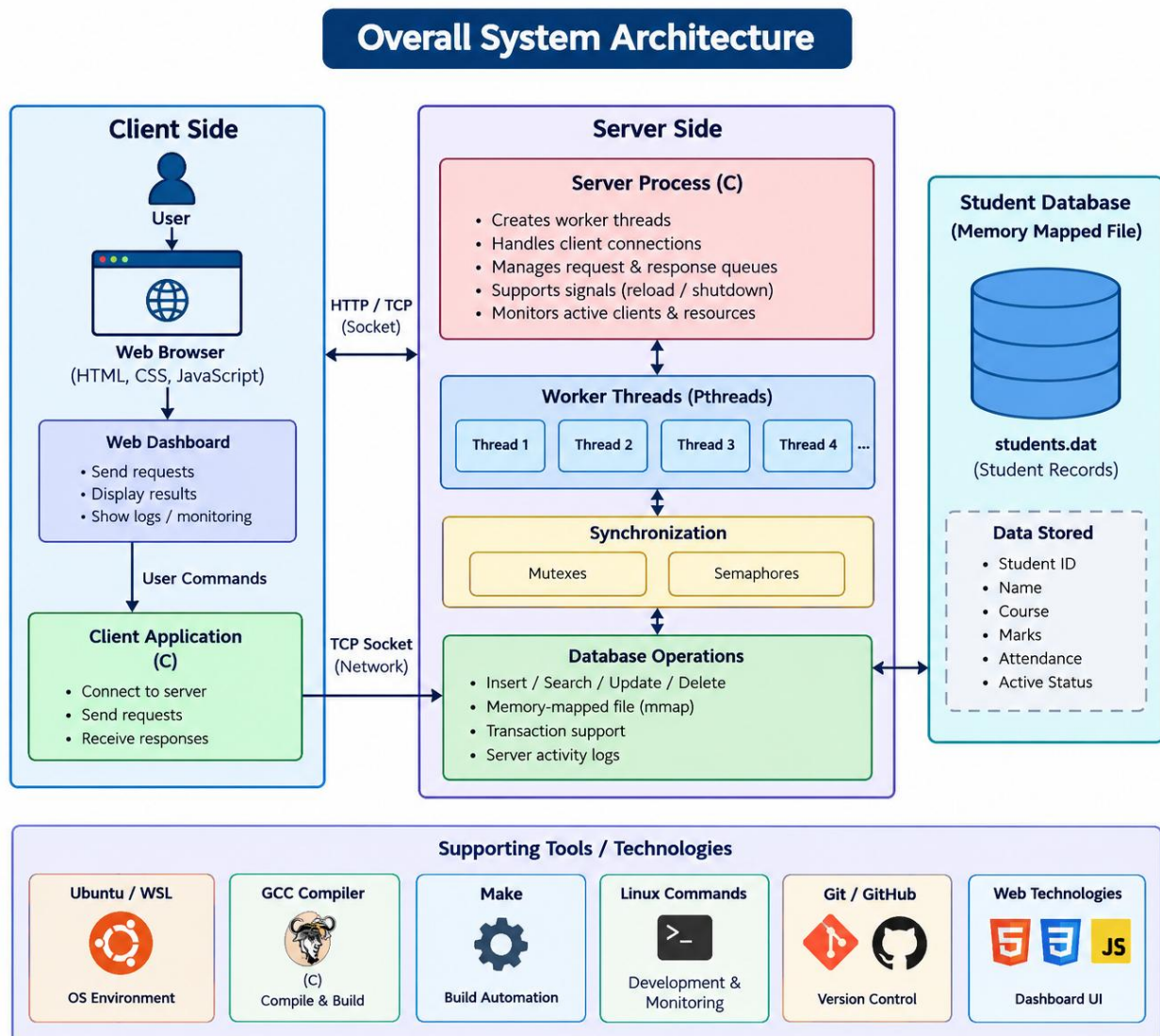
4.6 Other Tools / Libraries Used

The project uses several additional tools and libraries to support the development and execution of the Database Client-Server Management System.

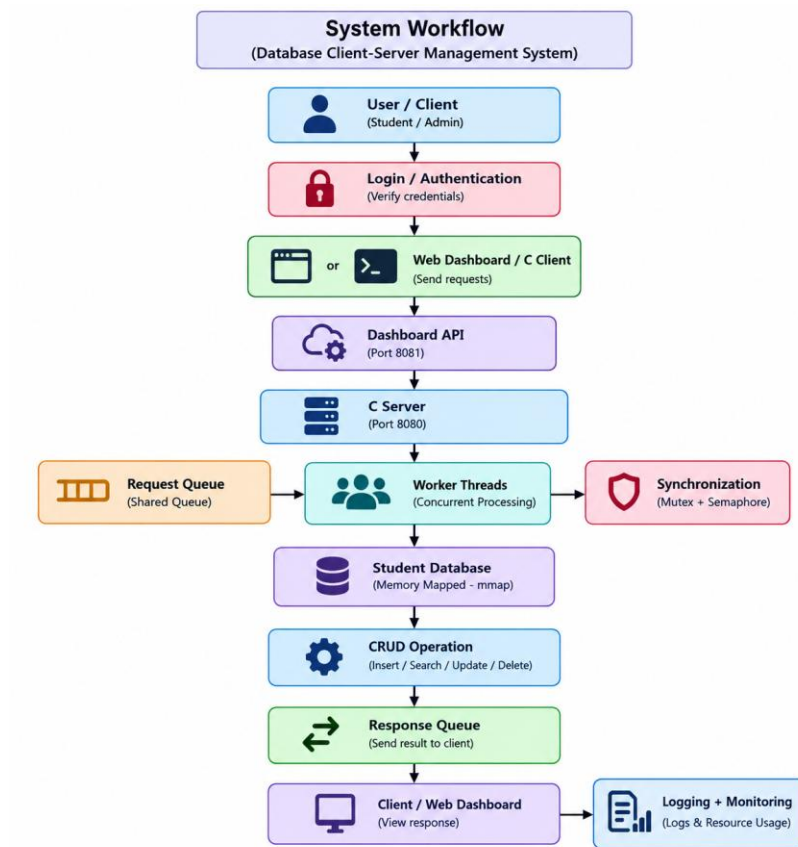
- **POSIX Library:** Provides APIs for threads, mutexes, semaphores, and other system-level operations.
- **Socket Library:** Used for communication between the client and server over TCP.
- **pthread Library:** Used to create and manage multiple worker threads.
- **Semaphore Library:** Used to synchronize access to shared queues and resources.
- **mmap Library:** Used for memory-mapped access to the student database.
- **Bootstrap:** Used to create a responsive and user-friendly web dashboard.
- **JavaScript Fetch API:** Used to communicate between the web dashboard and the dashboard API.
- **Git/GitHub:** Used for version control and maintaining the project source code.

CHAPTER -5 SYSTEM ARCHITECTURE AND DESIGN

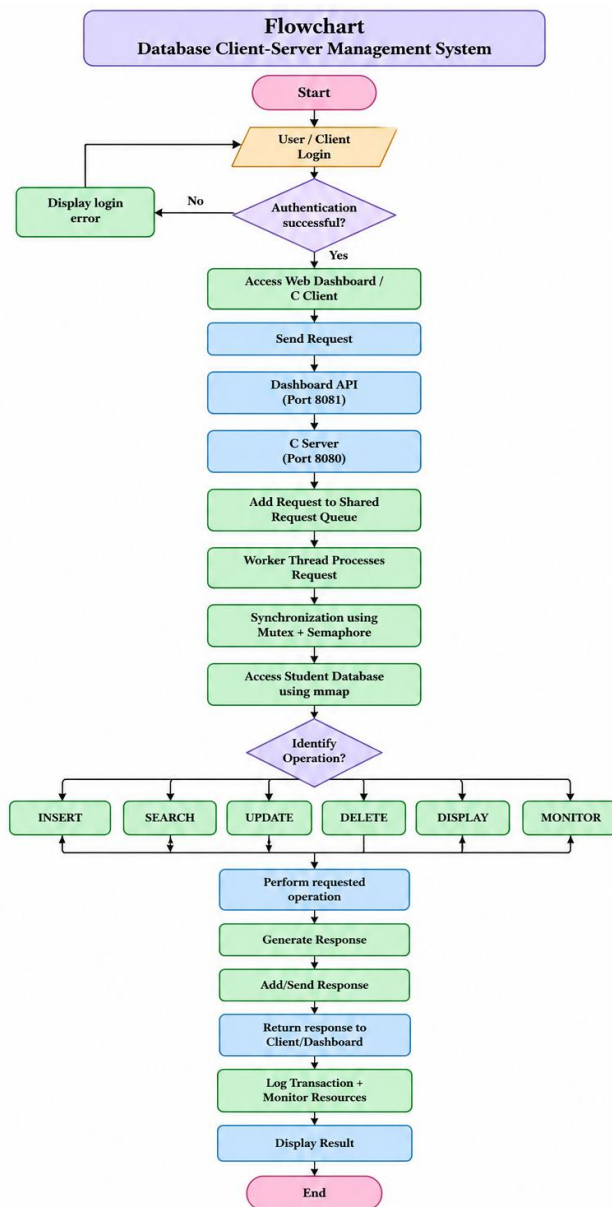
5.1 Overall System Architecture



5.2 System Workflow



5.3 Flowchart / Data Flow



5.4 Module Description

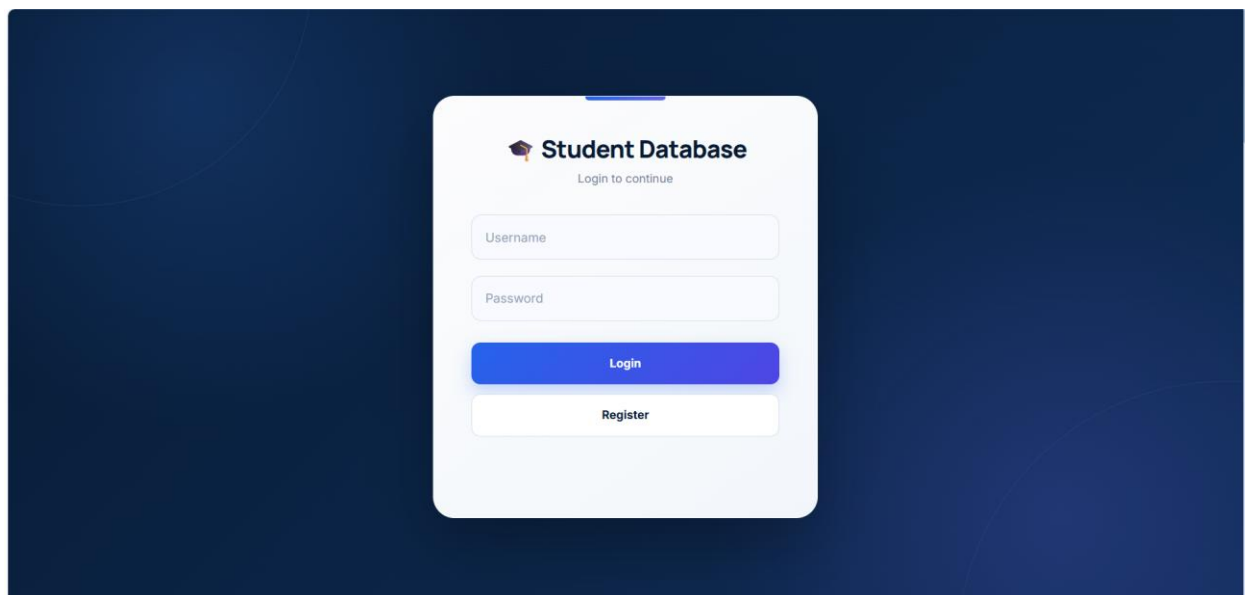
The Database Client-Server Management System is divided into several modules. Each module performs a specific function and works together to provide efficient and reliable student record management.

1. Server Module – Manages client connections, receives requests, and coordinates worker threads.
2. Client Module – Allows users to send commands such as INSERT, SEARCH, UPDATE, DELETE, and DISPLAY to the server.
3. Database Module – Stores and manages student records such as ID, name, course, marks, and attendance using memory mapping.
4. Queue Module – Maintains shared request and response queues for communication between clients and worker threads.

5. Thread and Synchronization Module – Uses POSIX threads, mutexes, and semaphores to support concurrent processing and prevent data conflicts.
6. Authentication Module – Provides user registration, login, logout, and session management to control access to the system.
7. Logging Module – Records server activities and database transactions for tracking and debugging.
8. Monitoring Module – Monitors active clients, worker threads, total threads, and memory usage.
9. Configuration Module – Stores and manages system settings such as server port, worker threads, maximum clients, and log level.
10. Web Dashboard Module – Provides a user-friendly interface for authentication, CRUD operations, student analytics, attendance alerts, recent activity, and system health.

Together, these modules provide a secure, concurrent, efficient, and user-friendly student database management system while demonstrating important operating system concepts.

5.5 Input and Output Design



Student Database Management

Client-Server Database Management System

Server Connected

Logout

TOTAL STUDENTS

6

Students in database

ACTIVE CLIENTS

1

Currently connected

WORKER THREADS

4

Request processing threads

TOTAL THREADS

7

Linux server threads

DATABASE INSIGHTS

Student Analytics

Live Data

Average Marks

94.2

Average Attendance

85.3%

Highest Marks

99.0

Attendance Alerts

1

ATTENTION REQUIRED

Attendance Alerts

Swathi

ID: 2

60%

SERVER ACTIVITY

Recent Activity

Student records loaded

6 records retrieved

Just now

User logged in

Welcome Swathi

Just now

Dashboard initialized

System ready

Just now

SYSTEM STATUS

System Health

Online

Database Server

Database

Worker Threads

Total Threads

Memory Usage

ONLINE

ACTIVE

4

7

2372 KB

Add Student

Student ID

Name

Course

CSE

Marks

Attendance

Add Student

Search Student

Student ID

414

Search

Student Found

ID: 414

Name: Yishnaani

Course: CSE

Marks: 98.00

Attendance: 88.00%

Update Student

Student ID

Marks

Attendance

Update Student

Delete Student

Student ID

Delete Student

Student Records

[Display Records](#)

ID	NAME	COURSE	MARKS	ATTENDANCE
1	Aswitha	CSE	98.00	90.00%
414	Yishnaani	CSE	98.00	88.00%
62	Priyadarshini	CSE	89.00	100.00%
6	Amulya	CSE	99.00	89.00%
19	Akshaya	CSE	88.00	90.00%
22	Srujana	ECE	92.00	95.00%

Server Monitoring

SERVER STATUS:

Running

PORT:

8080

WORKER THREADS:

4

DATABASE:

Memory Mapped

CHAPTER -6 IMPLEMENTATION

6.1 Module-wise Implementation

The project is divided into different modules, with each module performing a specific function. These modules work together to provide a complete and efficient client-server database management system.

1. Server Module

The server module manages client connections and receives requests. It creates worker threads and coordinates the overall operation of the system.

2. Client Module

The client module connects to the server and allows users to send commands such as INSERT, SEARCH, UPDATE, DELETE, and DISPLAY.

3. Database Module

The database module manages student records and performs CRUD operations. It uses memory mapping (mmap) for efficient database access.

4. Queue Module

The queue module manages the request and response queues. It stores client requests until worker threads are ready to process them.

5. Thread & Synchronization Module

This module manages POSIX worker threads, mutexes, and semaphores. It ensures that multiple clients can be handled safely without conflicts.

6. Authentication Module

The authentication module provides user registration, login, logout, and session management to control access to the system.

7. Logging Module

The logging module records important server activities and database transactions, helping in tracking system operations.

8. Monitoring Module

The monitoring module tracks active clients, worker threads, total threads, and memory usage to provide information about server performance.

9. Configuration Module

The configuration module reads and manages server settings such as port number, worker threads, maximum clients, and log level.

10. Web Dashboard Module

The web dashboard provides a user-friendly interface for managing student records and viewing analytics, attendance alerts, recent activities, and system health.

6.2 Algorithm / Pseudocode

The proposed system follows a client-server processing algorithm where client requests are received by the server, placed in a shared queue, and processed by worker threads. Synchronization mechanisms are used to safely access shared resources.

Main Algorithm: -

START

1. Load server configuration.
2. Initialize the student database using memory mapping.
3. Create the request and response queues.
4. Initialize mutexes and semaphores.
5. Create worker threads.
6. Create and bind the server socket.
7. Listen for incoming client connections.
8. When a client connects:
 - a. Accept the connection.
 - b. Authenticate the user.
 - c. Receive the client request.
 - d. Add the request to the request queue.
9. Worker thread takes a request from the queue.
10. Identify the requested operation:
 - IF INSERT → Add a new student record.
 - IF SEARCH → Find the required student record.
 - IF UPDATE → Modify the student record.
 - IF DELETE → Remove the student record.
 - IF DISPLAY → Display all student records.
 - IF MONITOR → Return server monitoring information.
 - IF LOGOUT → End the user session.
11. Use mutexes/semaphores when accessing shared resources.
12. Send the result back to the client.
13. Record the operation in the server log.
14. Continue accepting and processing requests.
15. On shutdown signal:
 - a. Stop accepting new clients.
 - b. Complete pending requests.

- c. Stop worker threads.
- d. Close sockets and release resources.
- e. Unmap the database.
- f. Shut down the server.

END

CRUD Processing Algorithm: -

START

Receive command from client

IF command = INSERT

 Validate student details

 Add record to database

 Send success response

ELSE IF command = SEARCH

 Search student ID

 If found, return student details

 Else return "Not Found"

ELSE IF command = UPDATE

 Find student ID

 Update marks and attendance

 Send success response

ELSE IF command = DELETE

 Find student ID

 Remove/deactivate record

 Send success response

ELSE IF command = DISPLAY

 Read all student records

 Send records to client

ELSE

 Send "Invalid Command"

END

This algorithm ensures that multiple client requests are handled efficiently while maintaining data consistency and secure access to the student database.

6.3 Core C / C++ Implementation

The core implementation of the project is developed mainly using the C programming language in the Ubuntu/WSL environment. C is used because it provides direct support for Operating System concepts such as threads, sockets, synchronization, memory mapping, and system calls.

The server module creates a TCP socket, accepts multiple client connections, and assigns requests for processing. Multiple POSIX worker threads are created to handle client requests concurrently. A shared request queue is used to manage incoming requests.

The project uses mutexes and semaphores to synchronize access to shared resources and prevent race conditions. The student database is implemented using a memory-mapped file through `mmap()`, allowing efficient access to student records.

The C implementation also includes separate modules for database management, request queues, authentication, configuration, logging, and system monitoring. CRUD operations are implemented for inserting, searching, updating, deleting, and displaying student records.

The server also supports signal handling for controlled shutdown and configuration reload. Logging functions record important server and database activities, while monitoring functions provide information about active clients, threads, and memory usage.

The main C modules are:

- `server.c` – Handles server operations and client connections.
- `client.c` – Sends requests and receives responses.
- `database.c` – Performs student database operations.
- `queue.c` – Manages request and response queues.
- `auth.c` – Handles user authentication.
- `logger.c` – Records system activities.
- `config.c` – Manages server configuration.
- `monitor.c` – Monitors server and system resources.

Overall, the C implementation combines networking, multithreading, synchronization, memory management, file handling, and system monitoring to create the complete client-server application.

6.4 Linux System Calls / APIs Used

The project uses various Linux system calls and POSIX APIs to implement networking, file handling, memory management, threading, and synchronization.

System Call / API	Purpose in the Project
socket()	Creates a TCP socket for client-server communication.
bind()	Assigns the server IP address and port to the socket
listen()	Makes the server ready to accept client connections
accept()	Accepts incoming client connections.
connect()	Allows the client to connect to the server.
send() / recv()	Sends and receives data between client and server.
open() / close()	Opens and closes database and log files.
read() / write()	Reads and writes data to files.
mmap() / munmap()	Maps and releases the student database in memory.
pthread_create()	Creates worker threads for concurrent request processing.
pthread_mutex_lock() / unlock()	Creates worker threads for concurrent request processing.
sem_wait() / sem_post()	Synchronizes access to shared queues and resources.
signal() / sigaction()	Handles server signals such as reload and shutdown.

These system calls and APIs allow the application to efficiently handle network communication, concurrent processing, synchronization, file operations, memory management, and server control.

6.5 Process / Thread Implementation

The project uses a multithreaded client-server architecture to handle multiple client requests efficiently. The server runs as the main process and continuously waits for incoming client connections.

When a client connects, the server accepts the connection and receives the client's request. The request is then placed into a shared request queue. A set of worker threads continuously monitors this queue and takes requests for processing.

The project uses POSIX threads (pthread) to create and manage the worker threads. Multiple worker threads can process different client requests concurrently, improving the responsiveness of the server.

Mutexes are used to protect shared data such as the request queue and database records. Semaphores are used to coordinate the availability of requests and ensure proper synchronization between the main server and worker threads.

The basic thread flow is:

Client → Server Process → Accept Client Request → Request Queue → Worker 1/2/3/4 → Database

Operation → Response to Client

The server is configured with 4 worker threads, allowing several requests to be processed simultaneously. When the server receives a shutdown signal, it stops accepting new connections, completes pending operations, terminates the worker threads, closes resources, and safely shuts down.

Thus, process and thread implementation enables the system to provide concurrent processing, efficient resource utilization, synchronization, and reliable handling of multiple clients.

6.6 IPC / Synchronization Implementation

The project uses inter-process and inter-thread communication mechanisms to allow clients, the server, and worker threads to communicate efficiently. Client and server communication is implemented using TCP sockets, through which requests and responses are exchanged.

Inside the server, a shared request and response queue is used to manage client requests. When a request is received, it is placed in the queue and a worker thread processes it. POSIX semaphores are used to coordinate the producer and worker threads, ensuring that workers process requests only when they are available.

Mutex locks are used to protect shared resources such as the database and request queue. This prevents multiple threads from modifying the same resource simultaneously and avoids race conditions.

Thus, the IPC and synchronization implementation provides safe communication, concurrent processing, data consistency, and efficient resource sharing between different components of the system.

6.7 Memory Management Implementation

The project uses memory mapping (mmap) for efficient management of the student database. The database file is mapped directly into the process's memory, allowing the server to access student records without repeatedly performing normal file read and write operations.

When the server starts, the database file is opened and mapped into memory. The required student records can then be accessed and modified directly through the mapped memory region. This improves data access efficiency and demonstrates the use of virtual memory and memory management concepts in Linux.

When the server is shutting down, the mapped memory is released using `munmap()`, and the required resources and file descriptors are properly closed.

The memory management flow is:

Database File → `open()` → `mmap()` → Memory-Mapped Region → Read / Update Student Records → `munmap()` → Release Memory

Thus, memory mapping provides efficient database access, reduced file I/O overhead, and practical implementation of Linux memory management concepts.

6.8 Optional User Interface Implementation

The project provides a simple web-based user interface to make the database system easy to use. The dashboard is developed using HTML, CSS, JavaScript, and Bootstrap. It connects to the C server through a small dashboard API.

The interface provides the following features:

- User Login and Registration for secure access.
- Student Record Management to insert, search, update, delete, and display records.
- Student Analytics such as average marks and attendance.
- Attendance Alerts to identify students with low attendance.
- Recent Activity to show recent database operations.
- System Health Monitoring to display active clients, worker threads, and memory usage.
- Logout to safely end the user session.

The web interface communicates with the backend server running on port 8080 through the dashboard API on port 8081. This provides a user-friendly way to interact with the C-based client-server system without directly entering commands in the terminal.

CHAPTER -7 EXECUTION AND DEMONSTRATION

7.1 Compilation in Ubuntu

The project is compiled and executed in the Ubuntu/Linux environment using GCC. The source files are organized into separate modules such as server, client, database, queue, authentication, logging, configuration, and monitoring.

The Makefile is used to simplify the compilation process. The following command is used

`make clean && make`

GCC compiles the C source files and links the required libraries. The `-pthread` option is used for POSIX thread support, and other required libraries are linked during compilation.

After successful compilation, the generated programs can be executed using:

`./server` and `./client`

In terminal 1: -

```
aswitha_patel@Aswitha:~$ cd ~/database_client_server
aswitha_patel@Aswitha:~/database_client_server$ make clean
rm -f server client
aswitha_patel@Aswitha:~/database_client_server$ make
gcc -Wall -Wextra -Iinclude -o server src/server.c src/database.c src/queue.c src/logger.c src/config.c src/monitor.c src/auth.c -pthread -lrt
gcc -Wall -Wextra -Iinclude -o client src/client.c -pthread -lrt
aswitha_patel@Aswitha:~/database_client_server$ ./server

===== SERVER CONFIGURATION =====
Port      : 8080
Workers   : 4
Max Clients : 20
Log Level  : 1
=====

=====
Database Client-Server Management System
=====
Server running on port 8080
Maximum clients: 20
Worker threads: 4
Waiting for client connections...

|
```

In terminal 2: -

```
aswitha_patel@Aswitha: ~/da x + v
aswitha_patel@Aswitha:~$ cd ~/database_client_server
aswitha_patel@Aswitha:~/database_client_server$ ./client
Connected to Database Server!
Server IP: 127.0.0.1
Server Port: 8080

Available commands:
REGISTER username password
LOGIN username password
LOGOUT
INSERT id name course marks attendance
SEARCH id
UPDATE id marks attendance
DELETE id
DISPLAY
MONITOR
EXIT

Enter request: |
```

The Dashboard API was started successfully on port 8081 using the `./dashboard_api` command.

The web interface was opened from the Ubuntu terminal using the `explorer.exe index.html` command. This launches the project dashboard in the Windows web browser.

In terminal 3: -

```
aswitha_patel@Aswitha: ~/da x + v
aswitha_patel@Aswitha:~$ cd ~/database_client_server
aswitha_patel@Aswitha:~/database_client_server$ ./dashboard_api
Dashboard API running on port 8081...
|
```

In terminal 4: -

```
aswitha_patel@Aswitha: ~/da x + v
aswitha_patel@Aswitha:~$ cd ~/database_client_server/dashboard
aswitha_patel@Aswitha:~/database_client_server/dashboard$ explorer.exe index.html
aswitha_patel@Aswitha:~/database_client_server/dashboard$ |
```

Thus, Ubuntu and GCC provide the complete environment required to compile, run, test, and debug the project.

7.2 Execution Commands

The project is executed in the Ubuntu/WSL terminal using the following commands:

```
cd ~/database_client_server
```

Compile the project:

```
make clean && make
```

Start the C server:

```
./server
```

In another terminal, start the Dashboard API:

```
cd ~/database_client_server
```

```
./dashboard_api
```

The Dashboard API runs on port 8081, while the main C server runs on port 8080.

To open the web dashboard:

```
cd ~/database_client_server/dashboard
```

```
explorer.exe index.html
```

The dashboard can then be accessed through the web browser.

7.3 Sample Inputs and Outputs

1. Register and Login

```
aswitha_patel@Aswitha:~$ cd ~/database_client_server
aswitha_patel@Aswitha:~/database_client_server$ ./client
Connected to Database Server!
Server IP: 127.0.0.1
Server Port: 8080

Available commands:
REGISTER username password
LOGIN username password
LOGOUT
INSERT id name course marks attendance
SEARCH id
UPDATE id marks attendance
DELETE id
DISPLAY
MONITOR
EXIT

Enter request: REGISTER Aswitha 0518
Server response: SUCCESS: Registration completed

Enter request: LOGIN Aswitha 0518
Server response: SUCCESS: Login successful. Welcome Aswitha!

Enter request:
```

2. Insert

```
Enter request: INSERT 414 Yishnaani CSE 95 90
Server response: SUCCESS: Student inserted

Enter request: |
```

3. Search

```
Enter request: SEARCH 414
Server response: SUCCESS: ID=414 Name=Yishnaani Course=CSE Marks=95.00 Attendance=90.00

Enter request: |
```

4. Update

```
Enter request: UPDATE 414 98 88
Server response: SUCCESS: Student updated

Enter request: |
```

5. Delete

```
Enter request: DISPLAY
Server response: ===== STUDENT DATABASE =====
ID: 1 | Name: Aswitha | Course: CSE | Marks: 98.00 | Attendance: 90.00%
ID: 414 | Name: Yishnaani | Course: CSE | Marks: 98.00 | Attendance: 88.00%
ID: 2 | Name: Swathi | Course: ECE | Marks: 90.00 | Attendance: 60.00%
ID: 45 | Name: Sushma | Course: CSE | Marks: 88.00 | Attendance: 90.00%
ID: 6 | Name: Amulya | Course: CSE | Marks: 99.00 | Attendance: 89.00%
ID: 19 | Name: Akshaya | Course: CSE | Marks: 88.00 | Attendance: 90.00%
ID: 22 | Name: Srujana | Course: ECE | Marks: 92.00 | Attendance: 95.00%
=====

Enter request: DELETE 45
Server response: SUCCESS: Student deleted

Enter request: DISPLAY
Server response: ===== STUDENT DATABASE =====
ID: 1 | Name: Aswitha | Course: CSE | Marks: 98.00 | Attendance: 90.00%
ID: 414 | Name: Yishnaani | Course: CSE | Marks: 98.00 | Attendance: 88.00%
ID: 2 | Name: Swathi | Course: ECE | Marks: 90.00 | Attendance: 60.00%
ID: 6 | Name: Amulya | Course: CSE | Marks: 99.00 | Attendance: 89.00%
ID: 19 | Name: Akshaya | Course: CSE | Marks: 88.00 | Attendance: 90.00%
ID: 22 | Name: Srujana | Course: ECE | Marks: 92.00 | Attendance: 95.00%
=====

Enter request: |
```

6. Server Monitoring

```
Enter request: MONITOR
Server response: SUCCESS: Monitoring information
Active Clients: 1
Max Clients: 20
Worker Threads: 4
Total Threads: 7
Memory Usage: 2236 kB

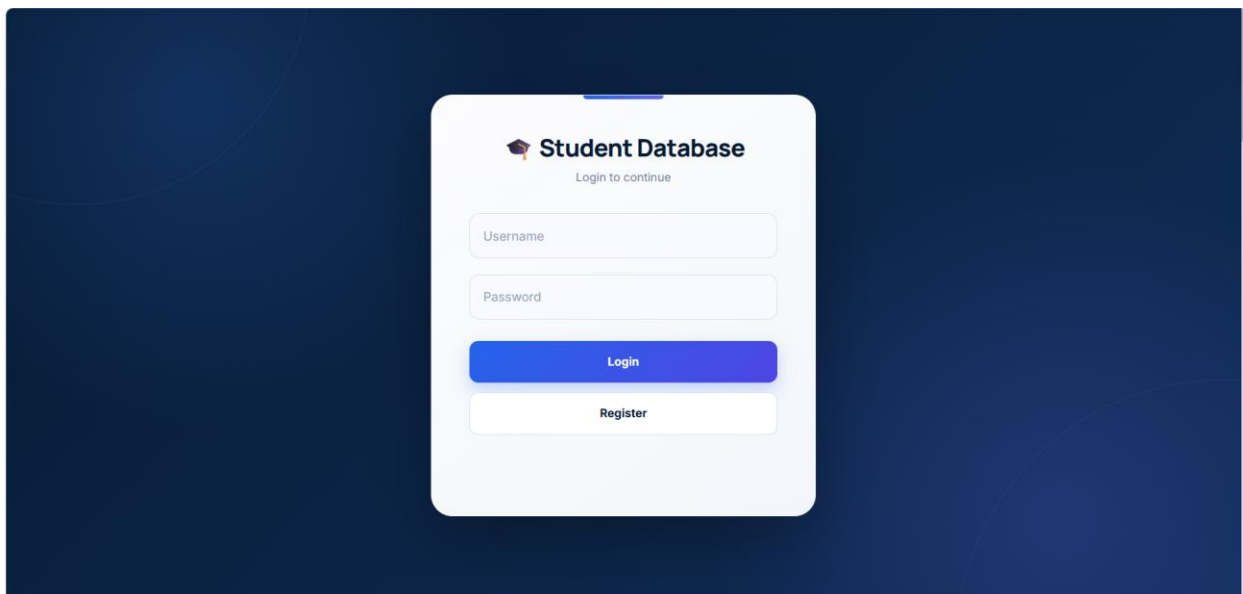
Enter request: |
```

7. Exit

```
Enter request: EXIT
Server disconnected.

Disconnected from server.
aswitha_patel@Aswitha:~/database_client_server$ |
```

7.4 Screenshots of Execution



Student Database Management

Client-Server Database Management System

Server Connected

Logout

TOTAL STUDENTS

6

Students in database

ACTIVE CLIENTS

1

Currently connected

WORKER THREADS

4

Request processing threads

TOTAL THREADS

7

Linux server threads

DATABASE INSIGHTS

Student Analytics

Live Data

Average Marks

94.2

Average Attendance

85.3%

Highest Marks

99.0

Attendance Alerts

1

ATTENTION REQUIRED

Attendance Alerts

Swathi

ID: 2

60%

SERVER ACTIVITY

Recent Activity

Student records loaded

6 records retrieved

Just now

User logged in

Welcome Swathi

Just now

Dashboard initialized

System ready

Just now

SYSTEM STATUS

System Health

Online

Database Server

Database

Worker Threads

Total Threads

Memory Usage

ONLINE

ACTIVE

4

7

2372 KB

Add Student

Student ID

Name

Course

CSE

Marks

Attendance

Add Student

Search Student

Student ID

414

Search

Student Found

ID: 414

Name: Yishnaani

Course: CSE

Marks: 98.00

Attendance: 88.00%

Update Student

Student ID

Marks

Attendance

Update Student

Delete Student

Student ID

Delete Student

Student Records					Display Records
ID	NAME	COURSE	MARKS	ATTENDANCE	
1	Aswitha	CSE	98.00	90.00%	
414	Yishnaani	CSE	98.00	88.00%	
62	Priyadarshini	CSE	89.00	100.00%	
6	Amulya	CSE	99.00	89.00%	
19	Akshaya	CSE	88.00	90.00%	
22	Srujana	ECE	92.00	95.00%	

Server Monitoring			
SERVER STATUS: Running	PORT: 8080	WORKER THREADS: 4	DATABASE: Memory Mapped

7.5 Demonstration of OS Concepts

The project demonstrates several important Operating System concepts through the implementation of the client-server database system.

- **Multithreading:** Four worker threads process client requests concurrently.
- **Process Management:** The server process manages multiple client connections.
- **Synchronization:** Mutexes protect shared database and queue resources from race conditions.
- **Semaphores:** Semaphores coordinate access to the request and response queues.
- **Inter-Process Communication:** TCP sockets are used for communication between clients and the server.
- **Memory Management:** `mmap()` is used to map the student database file into memory for efficient access.
- **File Management:** Student records, configuration files, and log files are stored and managed by the system.
- **Signal Handling:** Signals are used for controlled server shutdown and configuration reload.
- **Resource Monitoring:** The system monitors active clients, worker threads, total threads, and memory usage.
- **Authentication:** User login and registration provide controlled access to database operations.

Thus, the project provides a practical demonstration of concurrency, synchronization, memory management, IPC, file management, signal handling, and resource monitoring in Linux.

CHAPTER -8 CONCLUSION

8.1 Summary of the Project

The Database Client-Server Management System is a C-based application developed to manage student records efficiently in a Linux environment. The system provides operations such as insert, search, update, delete, and display of student information including ID, name, course, marks, and attendance.

The project supports multiple clients using TCP socket communication and POSIX threads. Worker threads process client requests concurrently, while mutexes and semaphores provide synchronization and prevent conflicts when accessing shared resources. The student database is managed using memory mapping (mmap) for efficient data access.

The system also includes user authentication, logging, signal handling, resource monitoring, and configuration management. A web-based dashboard provides a simple interface for performing database operations and viewing analytics, attendance alerts, recent activities, and server health information.

Overall, the project successfully demonstrates important Operating System concepts such as multithreading, synchronization, IPC, memory management, file handling, signal handling, and resource monitoring through a practical client-server application.

8.2 Objectives Achieved

The main objectives of the Database Client-Server Management System were successfully achieved through the implementation and testing of the project.

- A client-server architecture was developed for centralized student record management.
- CRUD operations were implemented for adding, searching, updating, deleting, and displaying student records.
- Multiple client support was achieved using POSIX threads and worker threads.
- Mutexes and semaphores were implemented to ensure safe access to shared resources.
- Memory mapping (mmap) was used for efficient database access.
- User authentication was implemented using registration, login, and logout features.
- Logging and monitoring were added to track server activities and system resources.
- Signal handling was implemented for controlled server shutdown and configuration management.
- A web dashboard was developed for easier interaction with the database system.
- The project successfully demonstrates important Operating System concepts in a practical application.

8.3 Results Achieved

The Database Client-Server Management System was successfully implemented and tested in the Ubuntu/WSL environment. The major results achieved are:

- Student records were successfully inserted, searched, updated, deleted, and displayed.
- Multiple client requests were handled using four worker threads.
- Mutexes and semaphores successfully provided synchronization for shared resources.
- The student database was successfully accessed using memory mapping (mmap).
- User authentication through registration and login was successfully implemented.
- Server activity and database operations were recorded through logging.
- The monitoring system successfully displayed active clients, worker threads, total threads, and memory usage.
- Signal handling enabled controlled server shutdown.
- The web dashboard successfully connected to the backend and displayed student records, analytics, alerts, and system information.
- The project successfully demonstrated the practical use of major Operating System concepts in a working client-server application.

Overall, the testing results show that the system provides efficient, concurrent, synchronized, and user-friendly student record management.

8.4 Skills and Learning Outcomes

Through the development of the Database Client-Server Management System, the following technical skills and learning outcomes were achieved:

- C Programming: Improved practical knowledge of C programming and modular program development.
- Linux Programming: Gained experience working with Ubuntu/WSL and Linux commands.
- Socket Programming: Learned how clients and servers communicate using TCP sockets.
- Multithreading: Learned to create and manage multiple worker threads using POSIX threads.
- Synchronization: Gained practical understanding of mutexes and semaphores for preventing race conditions.
- Inter-Process Communication: Learned how data is exchanged between clients and the server.
- Memory Management: Learned to use mmap() for memory-mapped file access.
- File Management: Gained experience in handling database, configuration, and log files.
- System Monitoring: Learned to monitor clients, threads, and memory usage.
- Web Development: Developed a basic dashboard using HTML, CSS, JavaScript, and Bootstrap.

- Problem Solving: Improved debugging, testing, and troubleshooting skills.
- Project Development: Gained experience in integrating different modules into a complete working system.

Overall, the project provided practical knowledge of Operating System concepts, system programming, networking, concurrency, synchronization, and web-based application development.

CHAPTER -9 REFERENCES

9.1 Books

1. Operating System Concepts – Abraham Silberschatz, Peter B. Galvin, Greg Gagne
2. The Linux Programming Interface – Michael Kerrisk
3. Computer Networks – Andrew S. Tanenbaum and David J. Wetherall
4. C Programming Language – Brian W. Kernighan and Dennis M. Ritchie
5. Advanced Programming in the UNIX Environment – W. Richard Stevens, Stephen A. Rago

9.2 Research Papers

1. “The Design and Implementation of a Modern Operating System” – Andrew S. Tanenbaum. Provides concepts related to operating system architecture, processes, threads, memory management, and resource management.
2. “An Overview of the Linux Operating System” – Provides an understanding of Linux system architecture, process management, memory management, and system calls.
3. “A Study of Client-Server Architecture and Its Applications” – Discusses client-server communication, request handling, networking, and distributed applications.
4. “Multithreading and Synchronization in Operating Systems” – Covers threads, mutexes, semaphores, concurrency, and prevention of race conditions.
5. “Memory-Mapped Files and Their Applications” – Discusses memory mapping techniques and efficient file access using memory-mapped regions.

These research materials helped in understanding the operating system, client-server communication, multithreading, synchronization, and memory management concepts used in the project.

9.3 Linux / GNU Documentation

1. GNU C Compiler (GCC) Documentation – Used for compiling and building the C source files.
2. GNU Make Documentation – Used for automating the compilation process through the Makefile.
3. Linux Man Pages – Used as a reference for Linux commands, system calls, and library functions.
4. Linux Kernel Documentation – Used for understanding Linux processes, threads, memory management, and system resources.
5. POSIX Threads (Pthreads) Documentation – Used for implementing worker threads, mutexes, and thread synchronization.
6. Linux mmap() Documentation – Used for implementing memory-mapped access to the student database.

7. Linux Socket Programming Documentation – Used for TCP socket creation, connection, communication, and client-server implementation.

These GNU/Linux documentation resources were used as references during the development, compilation, testing, and debugging of the project.

9.4 Online Resources

1. GeeksforGeeks – Used for references related to operating systems, C programming, threads, synchronization, and socket programming.
2. Linux Manual Pages (man pages) – Used to understand Linux system calls and commands such as `mmap()`, `pthread_create()`, `socket()`, `accept()`, and `signal()`.
3. GNU GCC Documentation – Used for compilation options, compiler commands, and debugging.
4. POSIX Documentation – Used for understanding threads, mutexes, semaphores, and other POSIX APIs.
5. W3Schools – Used as a reference for HTML, CSS, and JavaScript used in the web dashboard.
6. Bootstrap Documentation – Used for designing the responsive web dashboard interface.
7. GitHub Documentation – Used for version control, repository management, and project collaboration.

These online resources supported the development, implementation, testing, and documentation of the project.